

EXPERIMENT NO: 07

Experiment Title: To perform DOM traversal and manipulation, and to develop a simple To-Do List application with Add, Edit, and Delete functionalities using HTML, CSS, and JavaScript.

Software/Tools Required:

- Any text editor / VS Code
- Web Browser (Chrome/Edge) with Developer Console

Theory:

The Document Object Model (DOM) represents an HTML document as a tree of objects. JavaScript can access and modify these objects using DOM methods such as `getElementById()`, `querySelector()`, `createElement()`, and `appendChild()`. DOM manipulation allows web pages to be updated dynamically without reloading the page.

In a To-Do List application, JavaScript is used to create new task elements, modify existing tasks, and remove tasks from the DOM. The main operations performed are:

- Add: Creates and displays a new task.
- Edit: Modifies the text or properties of an existing task.
- Delete: Removes a task element from the DOM.
- DOM Traversal: Accesses and navigates through HTML elements to perform these updates.

Experiment Program Code

Code 1: Query Selector.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>querySelector() Example</title>

  <style>
    body{
      font-family: Arial, Helvetica, sans-serif;
      text-align: centre;
      margin-top: 50px;
    }

    .container{
      width : 500px;
      margin : auto;
      padding : 30px;
      border : 2px solid black;
      border-radius : 10px;
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>querySelector() Example</h1>
  </div>
</body>
</html>
```

```

        button{
            padding : 10px 20px;
            font-size: 16px;
            cursor : pointer ;
        }
    </style>
</head>
<body>
    <div class = "container">
        <h2 id = "heading">Javascript DOM Manipulation</h2>
        <p class = "message">
            This is the original message
        </p>
        <button onclick="changeContent()">
            Change Button
        </button>
    </div>

    <script>
        function changeContent(){
            let heading = document.querySelector('#heading');
            let message = document.querySelector('#message');

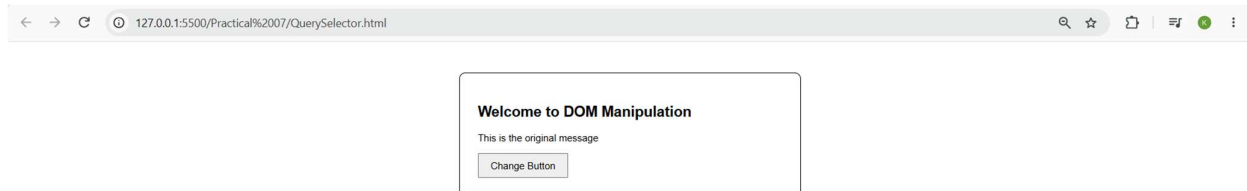
            heading.textContent = "Welcome to DOM Manipulation";

            message.textContent = "Content changed using querySelector()";

            heading.style.color = "blue";
            message.style.fontSize = "20px";
        }
    </script>
</body>
</html>

```

4. Output



5. Case Study Title

Design a To-Do List application using HTML, CSS and Javascript.

6. Case Study Program Code

Casestudy 06 Kanak Tembhurne 1290.html

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Smart To-Do List</title>

<style>
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
  }

  body {
    min-height: 100vh;
    background: #f6f1e8;
    color: #3f372f;
    padding: 35px 20px;
  }

  .container {
    max-width: 900px;
    margin: auto;
  }

  /* HEADER */

  header {
    text-align: center;
    margin-bottom: 30px;
  }

  header .icon {
    font-size: 45px;
    margin-bottom: 8px;
  }

  header h1 {
    font-size: 36px;
    color: #3d342d;
    margin-bottom: 8px;
  }

  header p {
    color: #8a7d70;
    font-size: 15px;
  }

  /* MAIN CARD */

  .main-card {
    background: white;
    border: 1px solid #e6ddd2;
    border-radius: 22px;
    padding: 28px;
    box-shadow: 0 10px 30px rgba(87, 69, 49, 0.08);
  }

  /* INPUT AREA */

  .input-area {
    display: flex;
    gap: 10px;
    margin-bottom: 25px;
  }

  #taskInput {
    flex: 1;
    padding: 14px 16px;
    border: 1px solid #ded3c6;
    border-radius: 11px;
    background: #fcfaf7;
    color: #3f372f;
    font-size: 15px;
    outline: none;
  }

  #taskInput:focus {

```

```

        border-color: #b58664;
        box-shadow: 0 0 3px rgba(181, 134, 100, 0.12);
    }

    #addBtn {
        padding: 14px 22px;
        border: none;
        border-radius: 11px;
        background: #9a6b4d;
        color: white;
        font-size: 15px;
        font-weight: bold;
        cursor: pointer;
    }

    #addBtn:hover {
        background: #81573e;
    }

    /* STATISTICS */

    .stats {
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        gap: 12px;
        margin-bottom: 25px;
    }

    .stat-box {
        background: #faf7f2;
        border: 1px solid #e8dfd5;
        border-radius: 14px;
        padding: 16px;
        text-align: center;
    }

    .stat-number {
        display: block;
        font-size: 25px;
        font-weight: bold;
        color: #9a6b4d;
        margin-bottom: 5px;
    }

    .stat-label {
        color: #8a7d70;
        font-size: 13px;
    }

    /* SECTION HEADER */

    .list-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 15px;
    }

    .list-header h2 {
        font-size: 21px;
    }

    #clearBtn {
        border: none;
        background: #f4e9df;
        color: #8a6045;
        padding: 8px 13px;
        border-radius: 8px;
        cursor: pointer;
        font-size: 13px;
    }

    #clearBtn:hover {
        background: #ead9c9;
    }

    /* TASK LIST */

```

```

#taskList {
  list-style: none;
}

.task-item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 14px;
  margin-bottom: 10px;
  background: #fcfaf7;
  border: 1px solid #e7ddd2;
  border-radius: 12px;
  transition: 0.2s;
}

.task-item:hover {
  border-color: #d2bda9;
  transform: translateY(-1px);
}

.check-btn {
  width: 22px;
  height: 22px;
  border-radius: 50%;
  border: 2px solid #b89576;
  background: white;
  cursor: pointer;
  flex-shrink: 0;
}

.task-text {
  flex: 1;
  font-size: 15px;
  color: #51463c;
  word-break: break-word;
}

.completed .task-text {
  text-decoration: line-through;
  color: #a59b91;
}

.completed .check-btn {
  background: #9a6b4d;
  border-color: #9a6b4d;
}

/* ACTION BUTTONS */

.actions {
  display: flex;
  gap: 6px;
}

.edit-btn,
.delete-btn {
  border: none;
  padding: 7px 10px;
  border-radius: 7px;
  cursor: pointer;
  font-size: 12px;
}

.edit-btn {
  background: #eee6dc;
  color: #755b46;
}

.delete-btn {
  background: #f5e4df;
  color: #a05d51;
}

.edit-btn:hover {
  background: #e3d6c7;
}

```

```

    }

    .delete-btn:hover {
        background: #ead0c9;
    }

    /* EMPTY MESSAGE */

    .empty {
        text-align: center;
        padding: 45px 20px;
        color: #a3978c;
    }

    .empty-icon {
        font-size: 40px;
        margin-bottom: 10px;
    }

    .empty p {
        font-size: 14px;
    }

    /* FOOTER */

    footer {
        text-align: center;
        margin-top: 20px;
        color: #a3978c;
        font-size: 12px;
    }

    /* RESPONSIVE */

    @media (max-width: 600px) {
        body {
            padding: 20px 12px;
        }

        .main-card {
            padding: 20px;
        }

        header h1 {
            font-size: 29px;
        }

        .input-area {
            flex-direction: column;
        }

        #addBtn {
            width: 100%;
        }

        .stats {
            grid-template-columns: 1fr;
        }

        .task-item {
            align-items: flex-start;
        }

        .actions {
            flex-direction: column;
        }
    }
</style>
</head>

<body>

<div class="container">

    <!-- HEADER -->

```

```

<header>
  <div class="icon">✓</div>
  <h1>Smart To-Do List</h1>
  <p>Manage your daily tasks using JavaScript DOM Manipulation</p>
</header>

<!-- MAIN -->

<div class="main-card">

  <!-- INPUT -->

  <div class="input-area">

    <input
      type="text"
      id="taskInput"
      placeholder="What do you need to do?"
    >

    <button id="addBtn" onclick="addTask()">
      + Add Task
    </button>

  </div>

  <!-- STATISTICS -->

  <div class="stats">

    <div class="stat-box">
      <span class="stat-number" id="totalTasks">0</span>
      <span class="stat-label">Total Tasks</span>
    </div>

    <div class="stat-box">
      <span class="stat-number" id="completedTasks">0</span>
      <span class="stat-label">Completed</span>
    </div>

    <div class="stat-box">
      <span class="stat-number" id="pendingTasks">0</span>
      <span class="stat-label">Pending</span>
    </div>

  </div>

  <!-- TASK HEADER -->

  <div class="list-header">

    <h2>My Tasks</h2>

    <button id="clearBtn" onclick="clearTasks()">
      Clear All
    </button>

  </div>

  <!-- TASK LIST -->

  <ul id="taskList">

    <div class="empty" id="emptyMessage">

      <div class="empty-icon">🚧</div>

      <p>
        No tasks yet. Add your first task!
      </p>

    </div>

  </ul>

</div>

```

```

<footer>
  JavaScript Lab Project • DOM Traversal • DOM Manipulation • Events
</footer>

</div>

<script>

  /* -----
    ADD TASK
  ----- */

  function addTask() {

    let input =
      document.getElementById("taskInput");

    let taskText =
      input.value.trim();

    // Check for empty task
    if (taskText === "") {

      alert("Please enter a task.");

      return;
    }

    // Create new list item
    let li =
      document.createElement("li");

    li.className = "task-item";

    // Create check button
    let checkButton =
      document.createElement("button");

    checkButton.className = "check-btn";

    checkButton.onclick =
      function() {

        toggleTask(li);

      };

    // Create task text
    let span =
      document.createElement("span");

    span.className = "task-text";

    span.textContent = taskText;

    // Create action container
    let actions =
      document.createElement("div");

    actions.className = "actions";

    // Edit button
    let editButton =
      document.createElement("button");

    editButton.className = "edit-btn";

    editButton.textContent = "Edit";

    editButton.onclick =
      function() {

        editTask(span);

      };

    // Delete button

```

```

    let deleteButton =
        document.createElement("button");

    deleteButton.className = "delete-btn";

    deleteButton.textContent = "Delete";

    deleteButton.onclick =
        function() {

            deleteTask(li);

        };

    // Add buttons to actions
    actions.appendChild(editButton);

    actions.appendChild(deleteButton);

    // Add everything to list item
    li.appendChild(checkButton);

    li.appendChild(span);

    li.appendChild(actions);

    // Add list item to task list
    document
        .getElementById("taskList")
        .appendChild(li);

    // Clear input
    input.value = "";

    input.focus();

    // Hide empty message
    document
        .getElementById("emptyMessage")
        .style.display = "none";

    updateStatistics();
}

/* -----
   COMPLETE / UNCOMPLETE TASK
   ----- */

function toggleTask(task) {

    task.classList.toggle("completed");

    updateStatistics();

}

/* -----
   EDIT TASK
   ----- */

function editTask(taskText) {

    let newText =
        prompt(
            "Edit your task:",
            taskText.textContent
        );

    if (newText !== null) {

        newText = newText.trim();

        if (newText !== "") {

            taskText.textContent =
                newText;

        } else {

```

```

        alert("Task cannot be empty.");
    }
}

/* -----
   DELETE TASK
   ----- */

function deleteTask(task) {
    task.remove();
    updateEmptyMessage();
    updateStatistics();
}

/* -----
   CLEAR ALL TASKS
   ----- */

function clearTasks() {
    let taskList =
        document.getElementById("taskList");

    // DOM traversal
    let tasks =
        taskList.querySelectorAll(".task-item");

    if (tasks.length === 0) {
        return;
    }

    if (confirm("Delete all tasks?")) {
        tasks.forEach(function(task) {
            task.remove();
        });
        updateEmptyMessage();
        updateStatistics();
    }
}

/* -----
   UPDATE EMPTY MESSAGE
   ----- */

function updateEmptyMessage() {
    let taskList =
        document.getElementById("taskList");

    let tasks =
        taskList.querySelectorAll(".task-item");

    let emptyMessage =
        document.getElementById("emptyMessage");

    if (tasks.length === 0) {
        emptyMessage.style.display =
            "block";
    }
}

```

```

    } else {

        emptyMessage.style.display =
            "none";

    }

}

/* -----
   UPDATE STATISTICS
   ----- */

function updateStatistics() {

    let taskList =
        document.getElementById("taskList");

    // DOM traversal
    let tasks =
        taskList.querySelectorAll(".task-item");

    let completed =
        taskList.querySelectorAll(
            ".task-item.completed"
        );

    let total =
        tasks.length;

    let completedCount =
        completed.length;

    let pending =
        total - completedCount;

    // Update DOM
    document.getElementById("totalTasks")
        .textContent = total;

    document.getElementById("completedTasks")
        .textContent = completedCount;

    document.getElementById("pendingTasks")
        .textContent = pending;

}

/* -----
   ENTER KEY SUPPORT
   ----- */

document
    .getElementById("taskInput")
    .addEventListener(
        "keypress",
        function(event) {

            if (event.key === "Enter") {

                addTask();

            }

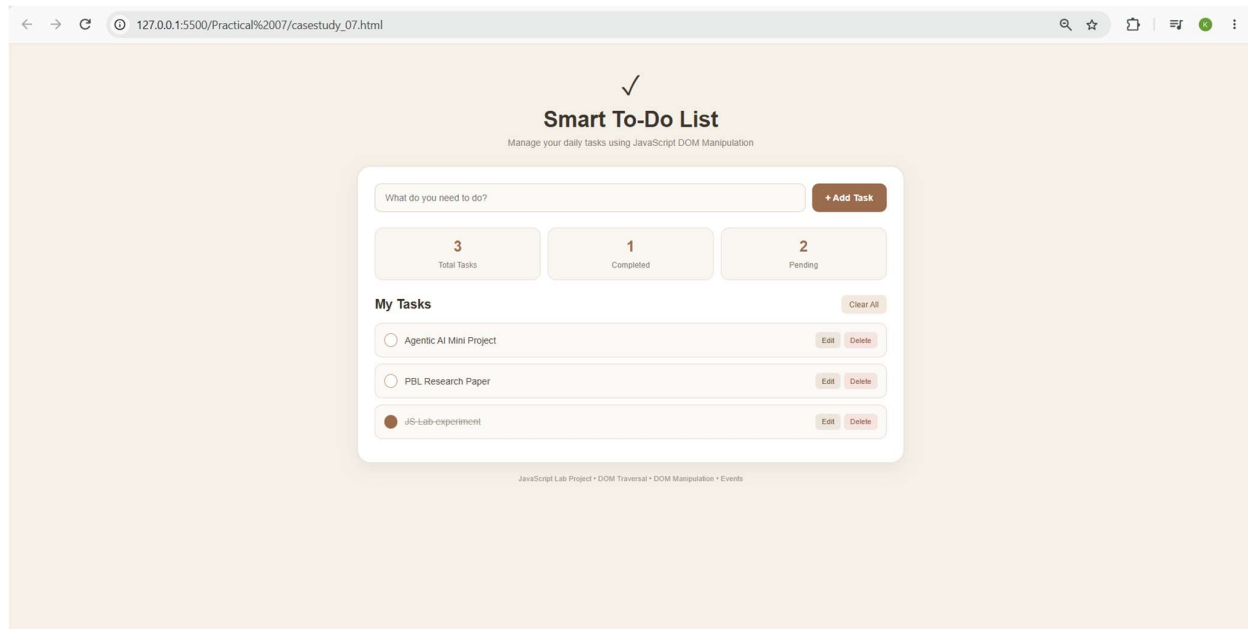
        }
    );

</script>
</body>
</html>

```

7. Output of the case study :

Name : Kanak Tembhurne PRN : 24070521290 File path :
[http://127.0.0.1:5500/Practical%2002/Casestudy_07_Kanak Tembhurne 1290.html](http://127.0.0.1:5500/Practical%2002/Casestudy_07_Kanak_Tembhurne_1290.html)



8. Conclusion

The To-Do List application was successfully developed using HTML, CSS, and JavaScript. DOM traversal and manipulation were performed to dynamically add, edit, and delete tasks from the webpage.